

Context Rot — warum KI in langen Sitzungen schlechter wird, und was wirklich dagegen hilft

Quellen: [video-summary-P031zpTqwhl.md](#), [video-summary-k2rkLm1eA9k.md](#),
[video-summary-slWwBfiuEsU.md](#), [video-summary-bz1C3dmiOvg.md](#),
[video-summary-IKHUKXp-nOA.md](#), [video-summary-gQeRjkb_Hlc.md](#),
[video-summary-SFtiPOTLBHA.md](#)

Bereits im [notes-audit-report.md](#) als Themen-Kandidat benannt: Sieben Videos im Repo berühren "Context Rot", meinen dabei aber teils unterschiedliche Mechanismen — und mehrere Einzeldateien warnen bereits explizit davor, sie zu verwechseln. Dieser Artikel bündelt das Phänomen selbst, grenzt es sauber von einem verwandten, aber anderen Effekt ab und fasst die über die Quellen verstreuten Gegenmaßnahmen zusammen.

Das eigentliche Phänomen: Antwortqualität sinkt mit der Tokenmenge

Context Rot bezeichnet den gemessenen, wiederholbaren Befund, dass mehr Input in einem Sprachmodell die Antwortqualität verschlechtert — **deutlich bevor** das technische Kontextfenster voll ist. Drei unabhängige Belege dafür stehen bereits im Repo:

- **Chroma-Studie** (18 getestete Modelle, u. a. Claude Sonnet 4, GPT-4.1, Qwen3-32B, Gemini 2.5 Flash): Jedes Modell baut mit wachsender Chat-/Materiallänge spürbar ab; steht relevante Information in der Mitte des Textes statt am Anfang/Ende, kann die Genauigkeit um 30 % oder mehr fallen — sauber strukturierter Text kann dabei teils *stärker* ablenken als chaotisch wirkender ([video-summary-P031zpTqwhl.md](#)).
- **"Context Arena"-Benchmark** (MRCR-v2): Bei 128.000 Token liegt die Trefferquote noch bei über 84 %, bei 1 Mio. Token fällt sie bei Claude Opus 4.6 auf 76 %, bei Gemini 3 Pro auf knapp 25 %, bei Gemini 2.0 Flash auf nur noch 10 % — per WebSearch exakt bestätigt (contextarena.ai) ([video-summary-IKHUKXp-nOA.md](#)).
- **Anthropic-Fellows-Studie** (Sam Martin, Fabien Roger): Modelle, die als Sicherheits-Monitor eine gefährliche Aktion erkennen sollen (Claude Opus 4.6, GPT 5.4, Gemini 3.1), übersehen sie 2- bis 30-mal häufiger, wenn zuvor rund 800.000 harmlose Tokens im Kontext standen ([video-summary-P031zpTqwhl.md](#)).

Wichtige Abgrenzung: nicht dasselbe wie "zu viele widersprüchliche Anweisungen"

[video-summary-slWwBfiuEsU.md](#) beschreibt explizit einen **anderen** Mechanismus, der unter ähnlichem Namen ("Claude wird dümmer") kursiert: Nicht die schiere Tokenmenge, sondern über Monate angesammelte, teils widersprüchliche Anweisungen (CLAUDE.md, Skills, Memory) zwingen das Modell, bei jeder Anfrage erst Konflikte aufzulösen, bevor es die eigentliche Aufgabe angeht. Beleg: Anthropic hat für Claude Opus 5 und Fable 5 über 80 % des Claude-Code-Systemprompts

gestrichen, ohne messbaren Leistungsverlust (Thariq Shihpar, Anthropic — per WebSearch bestätigter Blogpost, unabhängig auch in [video-summary-gQeRjkb_Hlc.md](#) verifiziert). Beide Mechanismen sind real, unabhängig voneinander belegt und verstärken sich in der Praxis wahrscheinlich gegenseitig (mehr Regeln = mehr Tokens im Kontext) — aber sie sind nicht dasselbe.

[video-summary-gQeRjkb_Hlc.md](#) führt Anthropics sechs "Then → Now"-Regeln für die Claude-5-Generation aus demselben Blogpost aus: weniger starre Regeln (mehr Urteilsvermögen), weniger Beispiele (klar designte Tool-Interfaces), Progressive Disclosure statt "alles vorab" (CLAUDE.md als Router statt Repository), weniger Wiederholung in Tool-Beschreibungen, automatische statt manueller Memory, reichhaltigere Referenzformate (z. B. HTML) statt reiner Markdown-Specs.

Gegenmaßnahmen (über die Quellen verstreut, hier gebündelt)

Gegen Context Rot im engeren Sinn (Tokenmenge):

- Faustregel: nur 40–50 % des Kontextfensters nutzen, Kontextverbrauch per Statusline im Blick behalten ([video-summary-IKHUKXp-nOA.md](#)).
- Für neue Themen einen neuen Chat starten statt endlos weiterzuschreiben; lange Verläufe zusammenfassen lassen und die Zusammenfassung als kompakten neuen Startpunkt verwenden ([video-summary-P031zpTqwhl.md](#)).
- **"Canary"-Trick** als Frühwarnsystem: eine winzige, leicht prüfbare Anweisung zu Beginn (z. B. "beende jede Antwort mit einem Smiley") — wird sie vergessen, sinkt meist schon die Gesamtqualität, bevor inhaltliche Fehler auffallen ([video-summary-P031zpTqwhl.md](#)).
- **Cross-Session Messaging** (Claude Code, ausgerollt Anfang August 2026, per offizieller Anthropic-Doku bestätigt — nur macOS/Linux inkl. WSL2, **nicht natives Windows**): Statt `/compact` (durchsucht das gesamte Gespräch, kostet selbst viele Token) eine neue, "leichte" Session starten und der alten nur eine kurze Zusammenfassung schicken lassen ([video-summary-bz1C3dmiOvg.md](#)).
- **Architekturansatz für Coding-Agents**: Recursive Language Models (RLM) laden lange Inputs (Dokument, Repository) nie vollständig in den Kontext, sondern legen sie als abfragbare Variable in einer Python-Session ab — das Modell "sieht" nur die Struktur, nicht den vollen Text, und ruft bei Bedarf rekursiv Kind-Agenten mit eigenem leerem Kontext auf. Laut MIT-Paper (Zhang, Kraska, Khattab, arXiv 2512.24601, per WebSearch bestätigt) 26 % besser als reine Kontext-Kompaktierung. Ausdrücklich **nicht** sinnvoll für kurze, einmalig beantwortete Dokumente ([video-summary-k2rkLm1eA9k.md](#)).

Gegen Regel-/Instruktions-Bloat (widersprüchliche Anweisungen):

- Claude Codes eingebautes `cclaude /doctor`-Kommando prüft fehlerhafte/doppelte Installationen, ungenutzte Skills/MCP-Server, Duplikate/Aufblähung in CLAUDE.md und verlangsamende Hooks, meldet Findings vor jeder Änderung ([video-summary-slWwBfiuEsU.md](#), [video-summary-gQeRjkb_Hlc.md](#)).
- Boris Chernys (Head of Claude Code, Anthropic) per WebSearch verifizierter Rat: alle sechs Monate gespeicherte Anweisungen löschen/archivieren und schauen, was das aktuelle Modell tatsächlich noch braucht — kein einmaliger radikaler Purge, sondern ein wiederkehrendes

Experiment ([video-summary-sIWwBfiuEsU.md](#); ergänzt [ai-agent-workflow.md](#) Punkt 7).

- Copy-paste-fähiger Zwei-Teile-Prompt (Audit: für jede gespeicherte Anweisung prüfen, ob sie ohne diese Anweisung ohnehin schon richtig gehandelt würde, eine inzwischen behobene Schwäche adressiert oder einer anderen Anweisung widerspricht; Rebuild: nur bewiesene Regeln behalten, den Rest archivieren statt löschen) — siehe [video-summary-sIWwBfiuEsU.md](#) für den vollen Wortlaut.

Einordnung: eine Stufe in einer größeren Engineering-Leiter

[video-summary-SFtiPOTLBHA.md](#) ordnet Context Engineering als zweite von fünf aufeinander aufbauenden Stufen ein (Prompt → Context → Harness → Loop → Graph Engineering, siehe auch [loop-engineering-ueberblick.md](#) für die dritte/vierte Stufe) — jede Stufe kompensiert laut Video eine damals aktuelle Modell-Schwäche, der Hebel für bessere Ergebnisse wandert dabei zunehmend von reiner Prompt-Formulierung hin zu Kontext-Kuratierung, Tooling und Orchestrierung.

Kernbotschaft

Context Rot ist ein empirisch mehrfach unabhängig bestätigtes Phänomen (Chroma, Context Arena, Anthropic Fellows Program) — Antwortqualität sinkt mit der Tokenmenge im Kontext, oft weit bevor das technische Limit erreicht ist. Es ist aber **nicht dasselbe** wie der ebenfalls real belegte, aber separate Effekt angesamelter widersprüchlicher Anweisungen, den Anthropic durch eine 80%ige Systemprompt-Kürzung für die Claude-5-Generation adressiert hat. Praktisch stehen für beide Probleme unterschiedliche, aber kombinierbare Werkzeuge bereit: für Tokenmenge Faustregeln, Zusammenfassen, Cross-Session-Messaging oder RLM-Architekturen; für Regel-Bloat periodische Audits (/doctor, Boris Chernys Sechs-Monats-Rhythmus).

Quelle: [context-rot-ueberblick.md](#)